

Format String Attacks: Arbitrary Read/Write

The bug

```
printf(user_input);      # WRONG – user controls the FORMAT string
printf("%s", user_input) # correct
```

With the format string you can walk the stack, read args, and write.

Reading with %n-family

`%p`, `%x` – read stack words

`%10$p` – read the 10th argument

`%s` – READ MEMORY AT an address you supply

`%n`, `%hn`, `%hhn` – WRITE number-of-printed-chars to an address

Arbitrary read recipe

```
payload = p32(target_addr) + "%7$s"  # print the string at target_addr
# leaks: any pointer (a GOT entry), is an informational goldmine
```

Arbitrary write recipe (GOT overwrite)

```
step 1: know an address you can point at (e.g., a GOT entry for exit() )
step 2: two %hn / four %hhn writes set the bytes of a function pointer
step 3: point the pointer to your function / ret2libc chain
```

Partial writes, width modifiers, and ordering get twitchy – pwntools

`fmtstr_payload` automates the grind in labs.

Why it still matters

Found in legacy C codebases constantly ('aha!' pattern in bug bounties)

Teaches *address arithmetic* and *pointer control* – ROP's foundation

Interplay: leak stack addresses → defeat ASLR → then ret2libc

Defensive takeaway (blue)

Never pass user data to printf(format) – the single-line fix

Compiler `__attribute__((format))` / `-Wformat=2` warns

A format bug is sabotage in a string: read anything, write anything.
Fixing them = validating that format is never attacker-controlled.

[← Ret2libc, ROP & Chains](#)

[Fuzzing, CVEs & Reading Adviso...](#)